Music & Video Player App

Kotlin · Jetpack Compose · Media3/ExoPlayer — Development Plan

This plan turns your project into a build order you can follow from a blank Android Studio project to a polished, installable app. It's organized in phases: each phase has a clear goal, a concrete deliverable, and the design tokens or code structures behind it. The visual language leans on glassmorphism and Material You dynamic color, but restrained — the album art and the music stay the focus, not the chrome around them.

Phase 0 — Environment & Project Setup (Days 1–2)

Get a clean, reproducible project skeleton before touching any UI.

- Install Android Studio (Koala/Ladybug or later) alongside your IntelliJ IDEA Ultimate setup — Compose Preview and Layout Inspector are Android-Studio-specific and worth having even if you write most code in IntelliJ.
- Create the project with an empty Compose Activity template, minSdk 26+ (covers Media3 features cleanly), Kotlin DSL for Gradle.
- Set up a version catalog (**libs.versions.toml**) pinning: Compose BOM, Media3 (media3-exoplayer, media3-session, media3-ui), Hilt, Room, Coil, Palette (androidx.palette) or Material Color Utilities.
- Initialize Git early; commit the empty scaffold, then commit after each phase below.
- Decide on the two SSDs: keep the project + emulator images on the internal 512GB SSD for speed; reserve the external 256GB enclosure drive for sample media libraries, exported APKs, and dataset-style test assets.

Phase 1 — Architecture Before Screens (Days 2–4)

Decide the skeleton so features slot in predictably instead of turning into spaghetti by week 3.

Layering

- **data** — Room database (library metadata, playlists, play history), MediaStore scanner for on-device audio/video, repository classes that expose Flow<T>.
- **domain** (optional but recommended) — use-case classes for things like 'get queue', 'toggle favorite', 'resolve now-playing colors'.
- **ui** — Compose screens + ViewModels (one ViewModel per screen, state exposed as StateFlow / UiState data classes).

Playback engine

- Use a foreground **MediaSessionService** (Media3) so playback survives navigation and shows system/notification controls.
- The Compose UI never talks to ExoPlayer directly — it goes through a MediaController bound to the session, so the mini-player, full player, and lock-screen controls all read the same source of truth.

- Wire Hilt for DI from day one: repository, player controller, and color-extraction service should all be injectable singletons.

Phase 2 — Design System Foundation (Days 4–6, before any real screens)

This is the step people skip and regret. Fifteen minutes here saves you from re-theming every screen later. Write these tokens into a single **Theme.kt** / **Dimens.kt** file.

Token	Value	Where it's used
Spacing scale	8 / 16 / 24 / 32 / 40 / 48 / 64 dp	All padding & gaps — never eyeball a margin
Corner radius (screen cards)	28 dp	Full player glass card, sheets
Corner radius (small cards)	24 dp	Album cards, list items
Corner radius (buttons)	Circle / 24 dp	Play button, chips
Blur radius	24–40 dp	Background artwork blur behind glass
Glass opacity	20–35%	Frosted panels over blurred art
Elevation	Very subtle (2–4dp shadows max)	Cards floating over background
Animation duration	250–350 ms, spring easing	Play button, queue sheet, crossfades
Type scale	Title 28–32sp Bold · Body 16sp Medium · Caption 13–14sp	Song title / artist / timestamps

Color strategy

- Base theme: Material 3 dynamic color (**dynamicLightColorScheme** / **dynamicDarkColorScheme** on Android 12+, with a static fallback scheme for older devices).
- Per-track accent: extract a dominant/vibrant color from the current album art using the Palette API (or Material Color Utilities for a Material-You-consistent swatch), then apply it to the progress bar, play button glow, and mini highlights only — not the whole screen, or it gets noisy.
- Keep a single dark neutral (something like a deep charcoal) as the base canvas so the accent color always has something calm to sit on top of.

Phase 3 — Build One Screen First: the Full Player (Week 2)

Resist the urge to scaffold ten screens. Get the Full Player right, because every other screen borrows its visual language and its color-extraction pipeline.

Layer stack (back to front)

- Blurred, slightly darkened album art filling the screen (graphicsLayer + RenderEffect blur, or a pre-blurred bitmap generated with RenderScript-replacement tooling for pre-Android-12 devices).
- Dark gradient scrim for text legibility.
- Glass card (20–35% opacity) holding the actual controls.
- Sharp, non-blurred album art on top of the glass card — the one clear, crisp element on the screen.
- Title / artist text, seek bar, transport controls, favorite/queue icons — in that visual weight order.

What 'done' looks like for this screen

- Play/pause morphs (not just swaps) between the two icon states.
- Seek bar drag updates position live and snaps back smoothly if released.
- Swiping the album art (or tapping skip) triggers a crossfade of both the blurred background and the accent color.
- This single screen should already feel like a finished app before you build anything else.

Phase 4 — Expand Outward From the Player (Weeks 3–4)

Screen	Core job	Notes
Mini player	Persistent bar above bottom nav	Shares element transition into Full Player on tap
Home / Library	Browse by playlist, artist, album, recently played	Reuse card + spacing tokens from Phase 2
Queue sheet	Reorderable up-next list	Slide up as a modal bottom sheet, glass background
Search	Fast local filter across titles/artists	Simple list — resist over-designing this one
Video player	Full-screen playback with minimal chrome	Transparent control bar, gesture-based brightness/volume
Settings	Theme, storage location, equalizer (optional)	Lowest design priority — build last

Build in this order — top to bottom of the table — so the highest-visibility screens (the ones you'll actually show people) are finished first.

Phase 5 — Media3 / ExoPlayer Integration Details (interleaved with Phase 3–4)

- **MediaSessionService** handles background playback, notification media controls, and Bluetooth/headset button events in one place — implement this before wiring the UI to avoid rework.
- Use **MediaController** from Compose (via a small wrapper ViewModel) to send play/pause/seek/skip commands and collect playback state as Flow.

- For video, decide early between Media3's **PlayerView** wrapped in an AndroidView, or a custom Compose surface — PlayerView is far less work and still themeable around the edges.
- Handle audio focus loss (calls, other apps) and headset unplug events — these are easy to forget and are the first things a real user will notice if missing.
- Cache thumbnails/album art with Coil so scrolling long lists doesn't re-decode images every frame.

Phase 6 — Motion & Polish Pass (Week 5)

Do this after every screen exists in a rough, functional state — polish once, across the whole app, rather than perfecting one screen while others stay bare.

- Shared-element-style morph from mini player to full player (Compose's **SharedTransitionLayout** if using a recent Compose version, or a manual size/position animation otherwise).
- Spring-based (not linear) animations for play button scale, queue sheet slide, and background crossfade.
- Subtle parallax or slow zoom on album art while a track plays — small motion cues that signal 'alive' without being busy.
- Audit every screen against the Phase 2 token table — anything not on the spacing/radius/blur scale gets fixed here.

Phase 7 — Testing, Performance & Release (Week 6)

- Test playback across interruptions: incoming calls, Bluetooth disconnects, screen rotation, app backgrounding.
- Profile blur performance on a mid-range device profile, not just your Dell-hosted emulator — blur and dynamic color extraction are the two most likely sources of jank.
- Check color-contrast on the extracted accent colors against your text — some album art produces accents too light or too saturated for legible text; add a contrast-safety clamp.
- Generate a signed release build, write a short Play Store listing, and do a final pass on app icon and splash screen.

Reference Apps to Study (Not Copy)

For each, ask specifically why a spacing choice, an animation curve, or a color decision works — the goal is understanding the reasoning, not the pixel values.

- **Apple Music** and **TIDAL** — glass panel restraint and typography hierarchy.
- **Plexamp** — arguably the best-designed player UI on Android; study its queue and now-playing transitions.
- **Poweramp** and **Nothing X** — clean, native-feeling Android aesthetics without borrowing from iOS.
- **Pixel Camera** — Material motion timing reference, unrelated to media but excellent for animation curves.

Suggested pace: roughly six weeks part-time alongside your Year 3 semester workload. Phases 0–2 are foundational and shouldn't be rushed — everything after them moves faster because the tokens and architecture are already decided.